ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		В.В. Жукалин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 5

Установка PostgreSQL и подключение Prisma ORM

по курсу: Web-программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321		Г.В. Буренков
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 Цель работы 3
2 Среда выполнения 4
3 Сервер, база, схема, роль и клиент 5
4 Создание роли и базы 6
5 Проверка подключения через psql 7
6 Установка Prisma ORM 8
7 Новые файлы проекта 9
8 Схема, окружение и клиент 10
9 Проверка validate, generate и TypeScript-запроса 12
10 Результаты lint и build 15
11 Возникшая проблема и диагностика 16
12 Ответы на контрольные вопросы 17
ЗАКЛЮЧЕНИЕ 19

1 Цель работы

Цель работы — установить и проверить локальный PostgreSQL, создать изолированную роль study_app и базу study_space для проекта PairWise, подключить Prisma ORM 6.19.2 к существующему Next.js-проекту, безопасно настроить строку подключения, сгенерировать Prisma Client и выполнить диагностический запрос к серверу базы данных. Физические таблицы предметной области в этой работе ещё не создаются.

2 Среда выполнения

Работа выполнена в Windows 10/11. Версии средств зафиксированы в таблице 1. Служба PostgreSQL — postgresql-x64-17, порт 5432. Каталог данных и установочный каталог заданы установщиком EDB по умолчанию. Установлены сервер PostgreSQL и Command Line Tools; pgAdmin доступен как дополнительный клиент и сервер не заменяет.

Таблица 1 — Версии среды

Компонент	**Версия**
Windows	10/11
Node.js	v24.20.0
npm	11.19.0
PostgreSQL	17.11
Порт	5432
Служба	postgresql-x64-17
prisma	6.19.2
@prisma/client	6.19.2

Листинг 1 — Вывод psql --version и версий Node/npm/Prisma

```
psql (PostgreSQL) 17.11
node v24.20.0
npm 11.19.0
Prisma schema loaded from prisma\schema.prisma
prisma                  : 6.19.2
@prisma/client          : 6.19.2
Computed binaryTarget   : windows
Operating System        : win32
Architecture            : x64
Node.js                 : v24.20.0
TypeScript              : 5.9.3
Query Engine (Node-API) : libquery-engine c2990dca591cba766e3b7ef5d9e8a84796e47ab7 (at
node_modules\@prisma\engines\query_engine-windows.dll.node)
PSL                     : @prisma/prisma-schema-wasm 7.1.1-
3.c2990dca591cba766e3b7ef5d9e8a84796e47ab7
Schema Engine           : schema-engine-cli c2990dca591cba766e3b7ef5d9e8a84796e47ab7
(at node_modules\@prisma\engines\schema-engine-windows.exe)
Default Engines Hash    : c2990dca591cba766e3b7ef5d9e8a84796e47ab7
Studio                  : 0.511.0
warn The configuration property `package.json#prisma` is deprecated and will be
removed in Prisma 7. Please migrate to a Prisma config file (e.g.,
`prisma.config.ts`).
For more information, see: https://pris.ly/prisma-config

Environment variables loaded from .env
```

3 Сервер, база, схема, роль и клиент

Сервер PostgreSQL — запущенная служба, которая слушает порт и обслуживает несколько баз. Кластер — набор баз одного экземпляра. База study_space — отдельное логическое пространство учебного проекта PairWise. Схема public — пространство имён объектов внутри этой базы. Роль study_app — учётная запись, от имени которой подключается приложение. Клиент psql отправляет SQL серверу. pgAdmin является графическим клиентом и не заменяет сервер.

4 Создание роли и базы

Под ролью postgres созданы прикладная роль без прав суперпользователя и отдельная база с владельцем study_app. Пароль роли в отчёт не помещён.

Листинг 2 — SQL создания роли и базы (пароль удалён)

```
CREATE ROLE study_app
  WITH LOGIN
  PASSWORD '<удалено>';

CREATE DATABASE study_space
  WITH OWNER = study_app
       ENCODING = 'UTF8';
```

5 Проверка подключения через psql

Подключение выполнено командой psql -h localhost -p 5432 -U study_app -d study_space. Запрос SELECT current_database(), current_user, inet_server_port() вернул study_space, study_app и 5432. На этом этапе \dt показывает отсутствие прикладных таблиц.

Листинг 3 — Результат прикладного SELECT

```
current_database | current_user | inet_server_port
------------------+--------------+------------------
 study_space      | study_app    |             5432
(1 ??????)
```

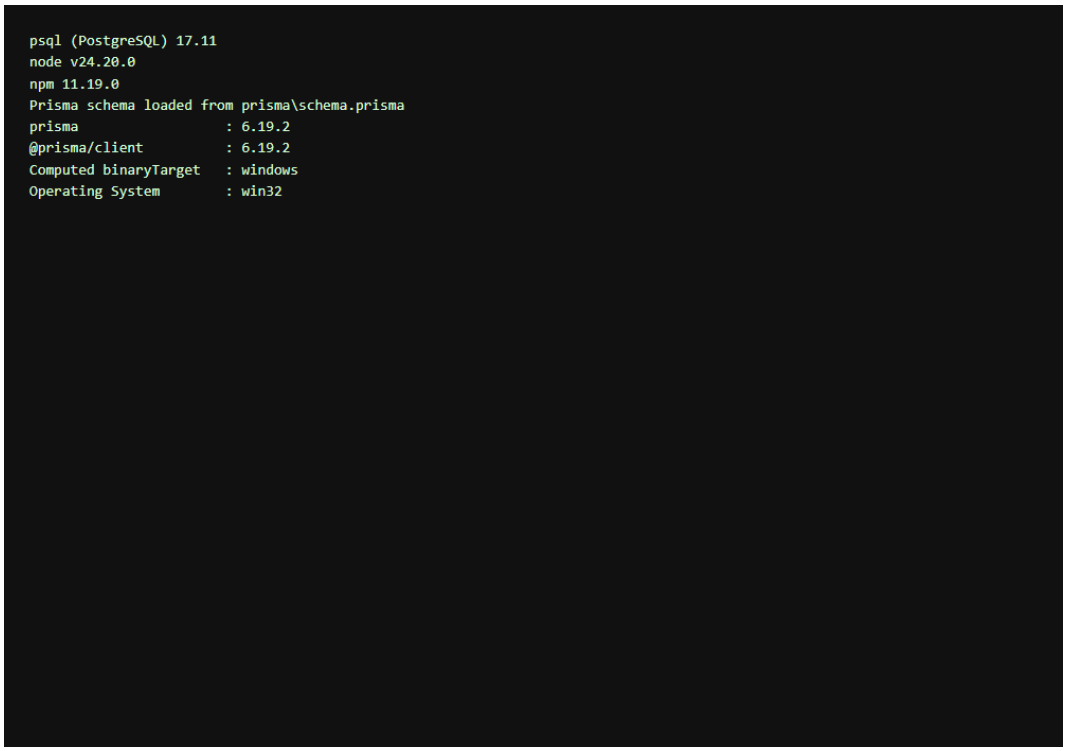


Рисунок 1 — Проверка psql и роли

6 Установка Prisma ORM

В корне проекта установлены согласованные версии prisma@6.19.2 и @prisma/client@6.19.2, а также dotenv и tsx. Команда npx prisma --version подтвердила совпадение CLI и клиента (листинг 1). Инициализация выполнена для провайдера postgresql.

7 Новые файлы проекта

Добавлены файлы конфигурации и проверки соединения. Дерево новых путей:

Листинг 4 — Дерево новых файлов

```
prisma/
  schema.prisma
app/lib/
  prisma.ts
scripts/
  check-prisma.ts
.env.example
.gitignore  (+ .env*)
```

8 Схема, окружение и клиент

В schema.prisma описаны generator client, datasource postgresql через env("DATABASE_URL") и временная модель SetupCheck только для проверки генерации клиента. Предметные таблицы появятся в следующей работе. Реальный .env исключён из Git правилом .env*; в репозиторий попадает только .env.example без секрета.

Листинг 5 — prisma/schema.prisma (ветка lr5)

```
// Генерирует Prisma Client для установленной версии Prisma ORM 6.19.2.
generator client {
  provider = "prisma-client-js"
}

// Читает локальное подключение PostgreSQL из DATABASE_URL.
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// Временная модель позволяет проверить генерацию клиента.
// В ЛР6 она удаляется до создания первой миграции.
model SetupCheck {
  id Int @id
}
```

Листинг 6 — .env.example

```
# Документирует формат подключения без настоящего пароля.
DATABASE_URL="postgresql://study_app:CHANGE_ME@localhost:5432/study_space?schema=publi
c"
```

Листинг 7 — app/lib/prisma.ts

```
import { PrismaClient } from "@prisma/client";

const globalForPrisma = globalThis as unknown as {
    prisma: PrismaClient | undefined;
};

export const prisma = globalForPrisma.prisma ?? new PrismaClient();

if (process.env.NODE_ENV !== "production") {
    globalForPrisma.prisma = prisma;
}
```

Листинг 8 — scripts/check-prisma.ts

```
import "dotenv/config";
import { prisma } from "../app/lib/prisma";

type ConnectionInfo = {
    database: string;
    role: string;
    port: number;
};
```

```
async function main() {
    const rows = await prisma.$queryRaw<ConnectionInfo[]>`
        SELECT
            current_database() AS database,
            current_user AS role,
            inet_server_port() AS port
    `;

    console.log("Prisma connected:", rows[0]);
}
```

9 Проверка validate, generate и TypeScript-запроса

Команды prisma format и prisma validate завершились успешно. Prisma Client сгенерирован. Скрипт db:check выполнил диагностический SELECT через Prisma Client и подтвердил study_space, study_app, порт 5432.

Листинг 9 — prisma validate

```
Prisma schema loaded from prisma\schema.prisma
The schema at prisma\schema.prisma is valid 🚀
warn The configuration property `package.json#prisma` is deprecated and will be
removed in Prisma 7. Please migrate to a Prisma config file (e.g.,
`prisma.config.ts`).
For more information, see: https://pris.ly/prisma-config

Environment variables loaded from .env
```

Листинг 10 — prisma generate

```
Prisma schema loaded from prisma\schema.prisma

✔ Generated Prisma Client (v6.19.2) to .\node_modules\@prisma\client in 55ms

Start by importing your Prisma Client (See: https://pris.ly/d/importing-client)

Tip: Want to turn off tips and other hints? https://pris.ly/tip-4-nohints

warn The configuration property `package.json#prisma` is deprecated and will be
removed in Prisma 7. Please migrate to a Prisma config file (e.g.,
`prisma.config.ts`).
For more information, see: https://pris.ly/prisma-config

Environment variables loaded from .env
```

Листинг 11 — Результат TypeScript-запроса

```
Prisma connected: { database: 'study_space', role: 'study_app', port: 5432 }
```

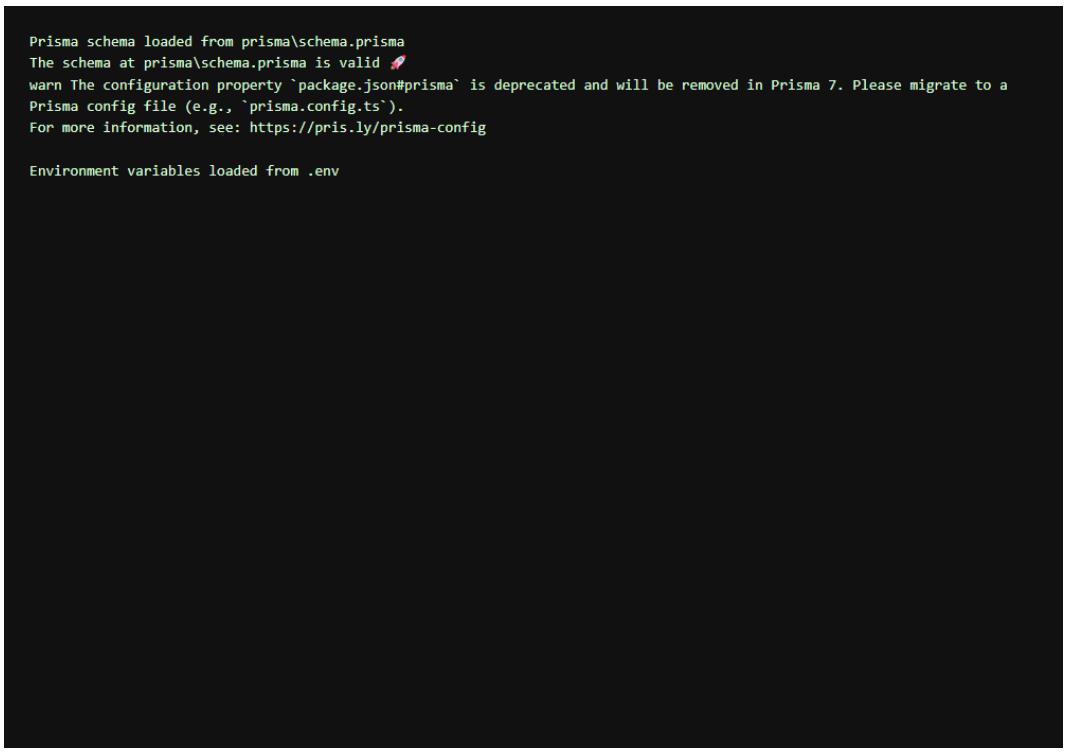


Рисунок — prisma validate

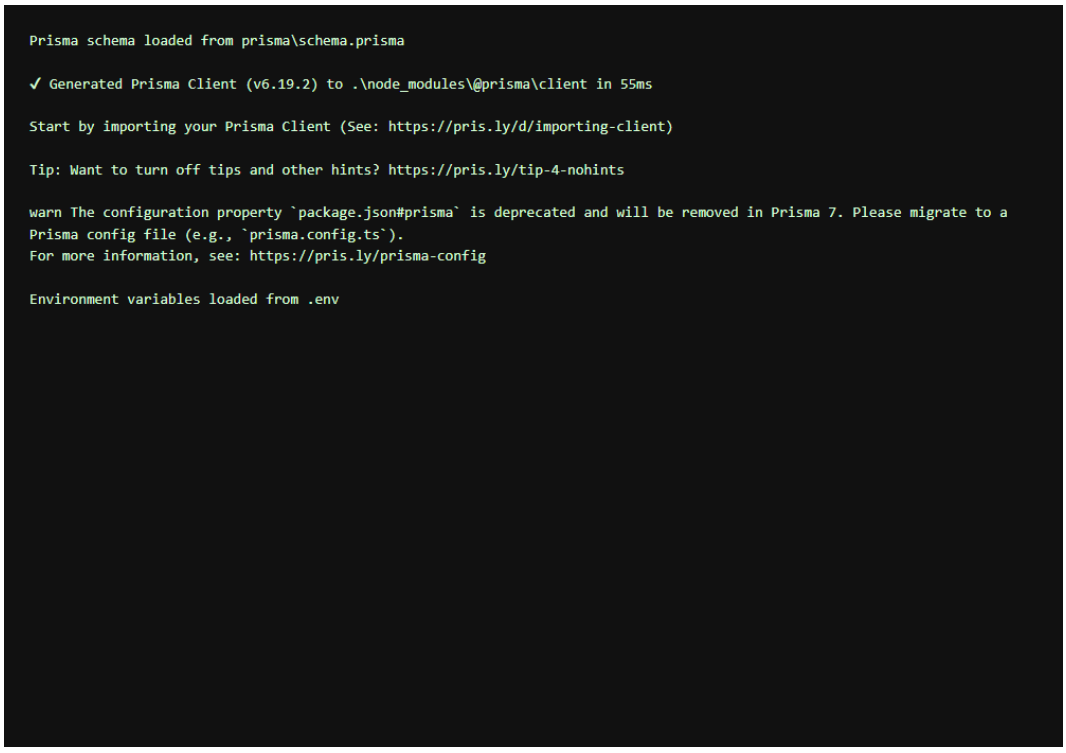


Рисунок — prisma generate

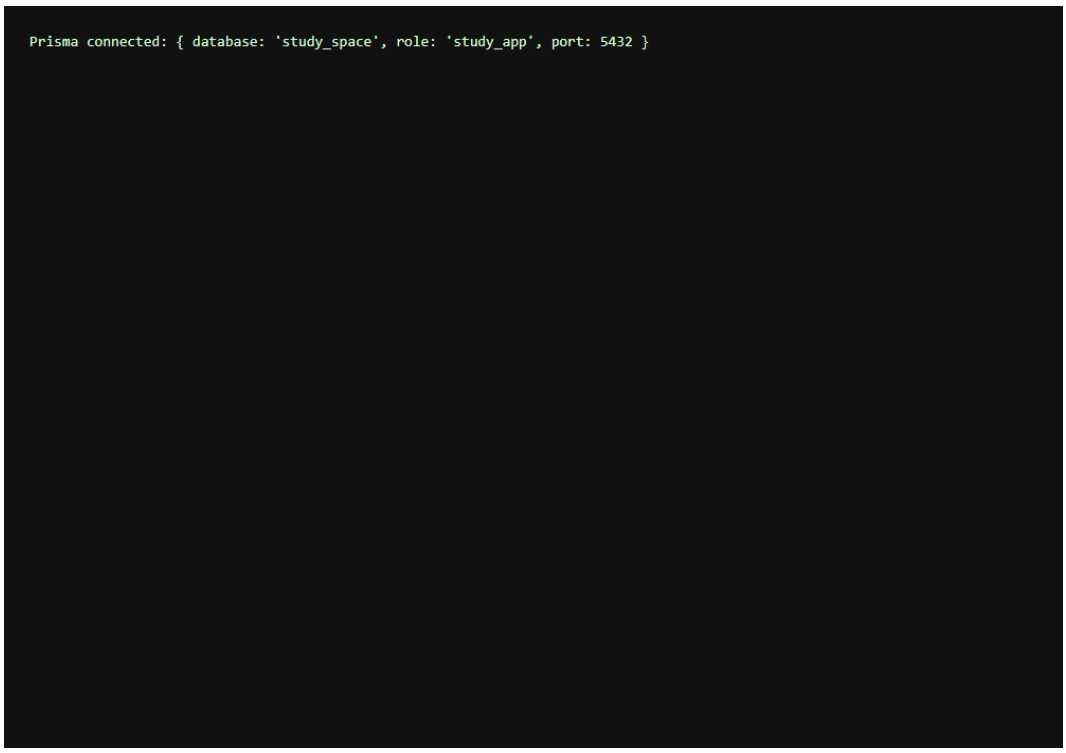


Рисунок — TypeScript-запрос

10 Результаты lint и build

Команда npm run lint завершилась без ошибок ESLint (имеется предупреждение о deprecated-правиле jsx-indent плагина stylistic, на результат проверки не влияет). Команда npm run build завершилась без ошибок.

Листинг 12 — npm run lint

```
> pairwise@0.1.0 lint
> eslint

[@stylistic/eslint-plugin]: You are using deprecated rule("jsx-indent"), please use
"indent" instead.
```

11 Возникшая проблема и диагностика

После установки PostgreSQL команда psql в уже открытом окне PowerShell не находилась в PATH. Диагностика: where.exe psql не возвращал путь, тогда как SQL Shell из меню Пуск работал. Решение — открыть новый сеанс PowerShell после обновления пользовательского PATH каталогом bin установленного PostgreSQL 17 и повторить psql --version. Ошибка подключения к неверной роли или порту исключалась проверкой \conninfo и SELECT current_database(), current_user, inet_server_port().

12 Ответы на контрольные вопросы

Чем сервер PostgreSQL отличается от базы данных? Сервер — процесс и служба, принимающие подключения и управляющие файлами данных. База данных — одно логическое пространство внутри сервера; один сервер обслуживает несколько баз.

Что такое кластер баз данных PostgreSQL? Это набор баз, управляемых одним экземпляром сервера (общие каталог данных, настройки и порт).

Для чего нужна роль PostgreSQL? Роль задаёт учётную запись и права доступа. Приложение PairWise подключается как study_app, а не как администратор.

Почему приложение не должно работать под ролью postgres? Роль postgres имеет избыточные административные права. Ошибка приложения под ней опаснее, чем под ограниченной прикладной ролью.

Чем база данных отличается от схемы public? Клиент сначала подключается к базе study_space, а уже внутри неё обращается к объектам схемы (здесь public). База и схема — разные уровни пространства имён.

Какие параметры использует psql для подключения? Ключевые параметры: -h хост, -p порт, -U роль, -d база. В работе: localhost, 5432, study_app, study_space.

Что показывает \conninfo? Метакоманда выводит сведения о текущем соединении: база, пользователь, хост и порт.

Из каких частей состоит Prisma ORM? Из Prisma Schema, CLI (validate, generate, migrate), Prisma Client для TypeScript и (при миграциях) движка миграций.

Что описывают datasource и generator? datasource задаёт провайдер и URL подключения. generator указывает, какой клиент генерировать (здесь prisma-client-js).

Для чего используется DATABASE_URL? Это строка подключения PostgreSQL для Prisma: роль, хост, порт, база и schema=public.

Почему DATABASE_URL нельзя объявлять с NEXT_PUBLIC_? Префикс NEXT_PUBLIC_ попадает в клиентский бандл. Секрет подключения не должен быть доступен браузеру.

Почему специальные символы пароля могут потребовать percent-encoding? В URL символы вроде @, :, /, % имеют особое значение. Их кодируют percent-encoding, иначе строка подключения разбирается неверно.

Почему версии prisma и @prisma/client должны совпадать? CLI генерирует клиент под конкретную версию API. Расхождение версий приводит к несовместимому клиенту и ошибкам выполнения.

Что делает prisma validate? Проверяет синтаксис и согласованность schema.prisma без изменения базы.

Что делает prisma generate? Генерирует Prisma Client в node_modules по текущей схеме.

Почему PrismaClient сохраняют в globalThis в development? Чтобы при горячей перезагрузке Next.js не создавать новый экземпляр на каждый reload и не исчерпывать пул соединений.

Чем migrate dev отличается от migrate deploy? migrate dev создаёт и применяет миграции в разработке. migrate deploy применяет уже существующие миграции, обычно в production.

Почему в этой работе миграция не выполняется? Физические таблицы предметной области проектируются в ЛР6. Здесь проверяются только установка, конфигурация, генерация клиента и соединение.

Что означают типовые ошибки подключения P1000 и P1001? P1000 — отказ аутентификации (неверные учётные данные). P1001 — сервер недоступен (не запущен, неверный хост или порт).

Какие данные нельзя помещать в отчёт и репозиторий? Реальные пароли ролей, DATABASE_URL с паролем, AUTH_SECRET, содержимое cookie/JWT и любые другие секреты.

ЗАКЛЮЧЕНИЕ

Установлен PostgreSQL 17.11 (служба postgresql-x64-17, порт 5432). Созданы роль study_app и база study_space. Подключена Prisma ORM 6.19.2 с совпадающим @prisma/client. Строка подключения хранится локально, .env.example без секрета. Временная модель SetupCheck позволила проверить generate; предметные таблицы не создавались. Диагностический TypeScript-запрос подтвердил study_space | study_app | 5432. Команды lint и build завершились без ошибок.